

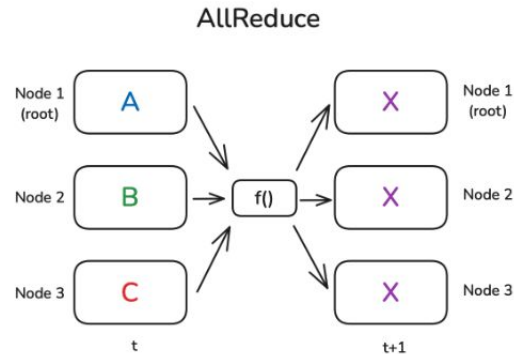
PIPELINE PARALLELISM

GPU clusters

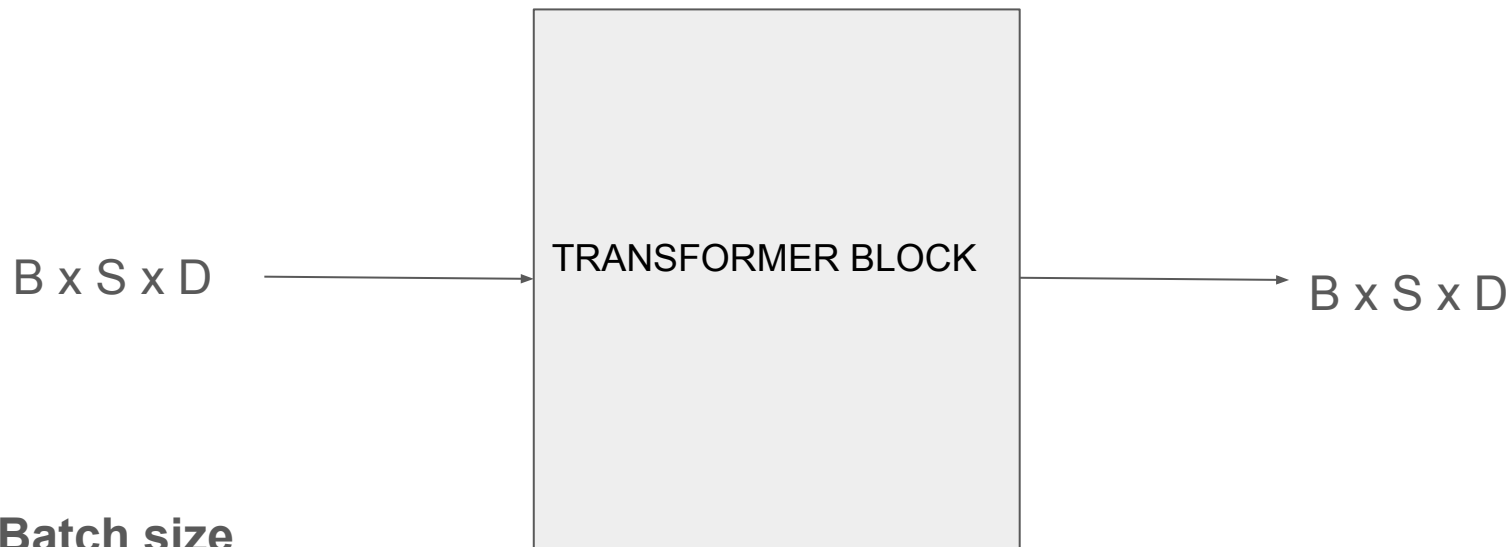
- *Llama 3 405B is trained on up to 16K H100 GPUs, each running at 700W TDP with 80GB HBM3, using Meta's Grand Teton AI server platform*
- A DGX cluster has 8 H100 connected by SXM (NVLink). Bandwidth is around 600-900 GB/s
- To connect GPUs across clusters, you need PCIe which has reduced bandwidth. Bandwidth is around 64 GB/s

Other parallelism with low bandwidth transfer

- Both data parallelism (DP) and tensor parallelism (TP) require all-reduce communications
- DP requires it for syncing gradients and optimizers
- TP requires it for syncing partial outputs (row-parallel)
- **Syncing weights, gradients and partial outputs with PCIe will have a lot of communication overhead**



Pipeline parallelism communication footprint

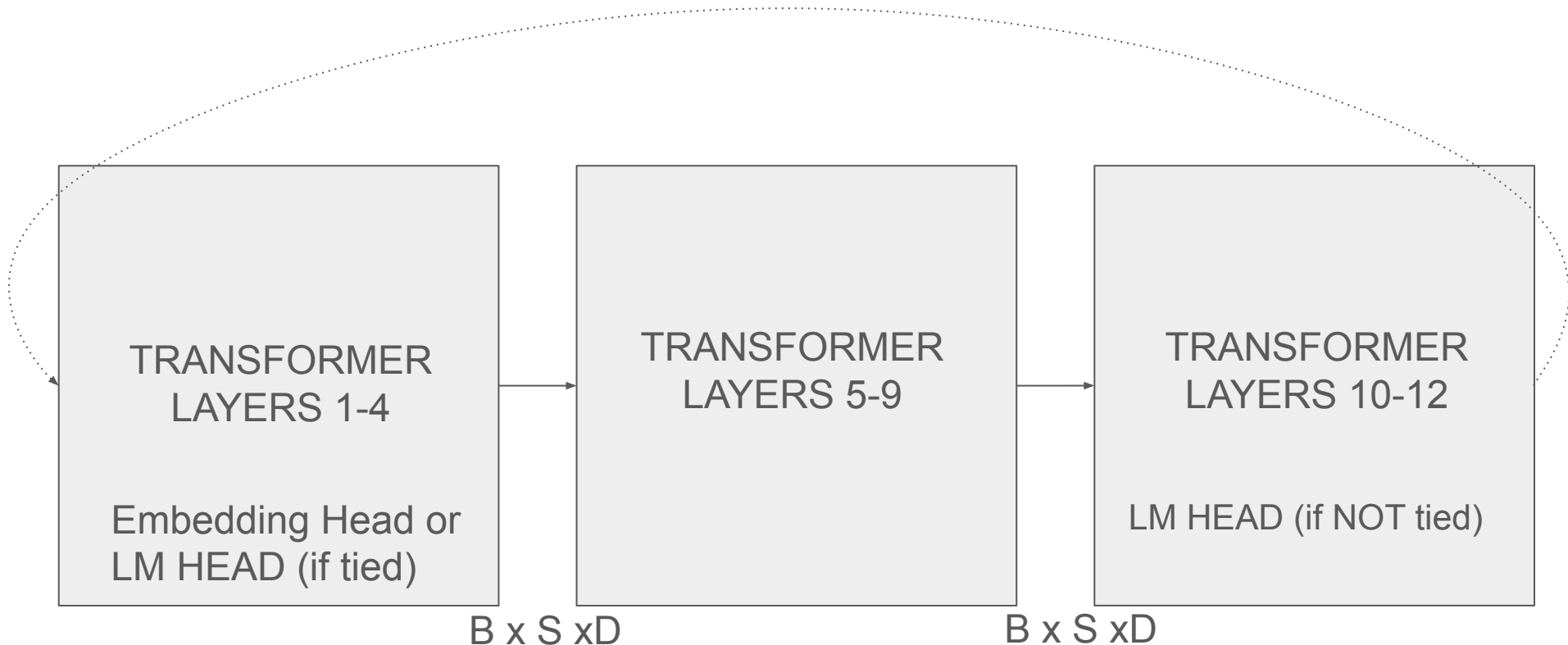


B: Batch size

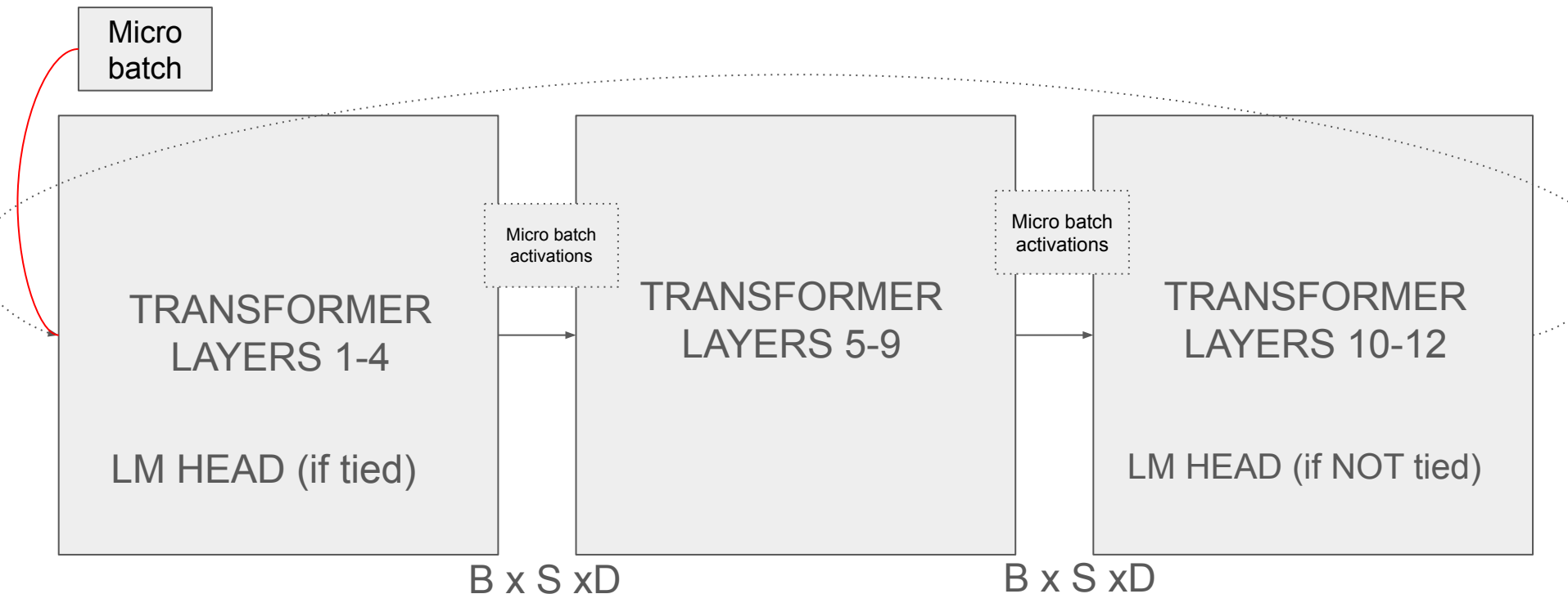
S: Sequence length. These are list of sequence indexes from embedding table (integers)

D: Hidden dimension: Typically from 512-16384 dimension

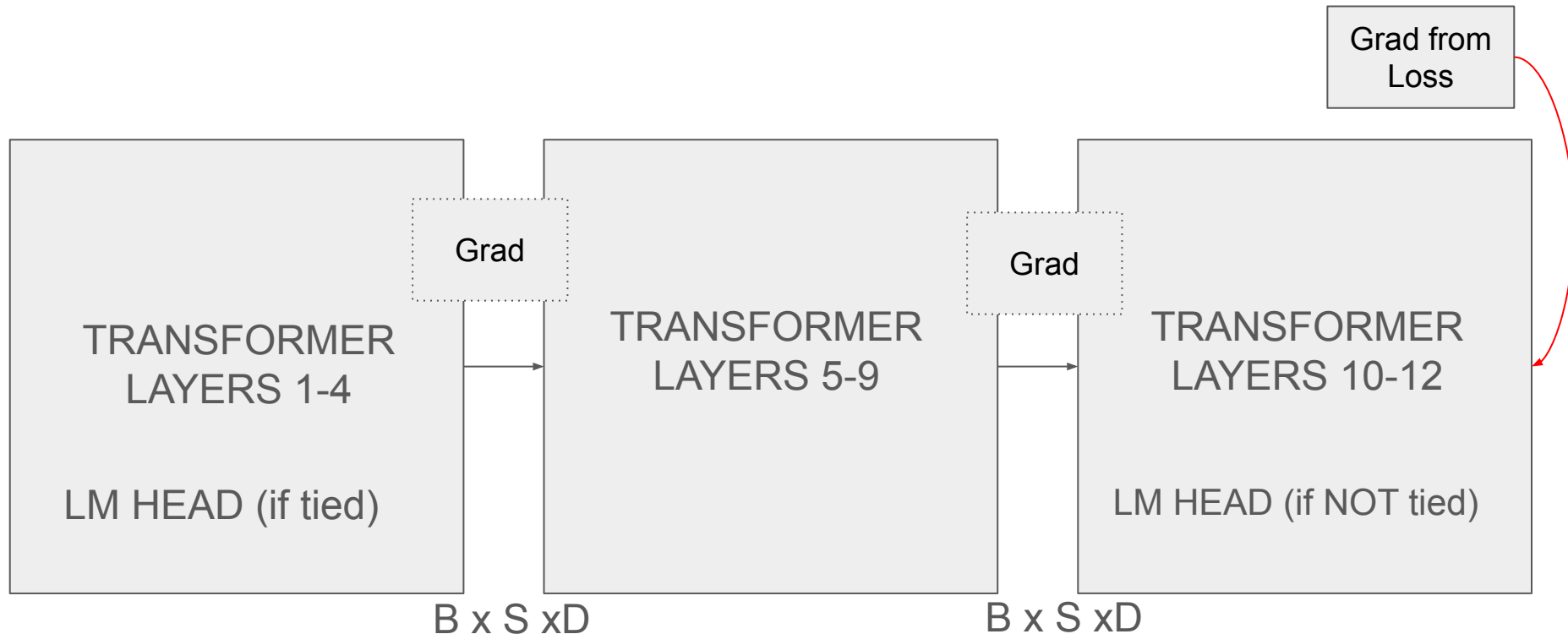
PIPELINE PARALLELISM MODEL SPLIT



PIPELINE PARALLELISM MODEL SPLIT (FORWARD)



PIPELINE PARALLELISM MODEL SPLIT (BACKWARD)



WHY PIPELINE PARALLELISM

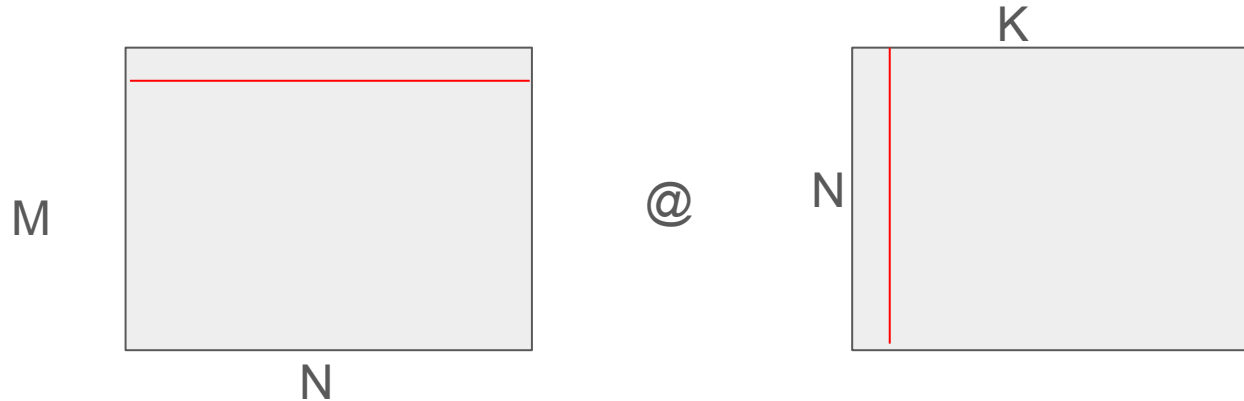
- **Low communication overhead**
- **Works well with homogenous architectures (same shape input and outputs like in a transformer)**
- **Empirically it has been shown that depth (number of layers in transformer) is better for generalization as compared to width (weights dimension)**

Let's cover some basics

- FLOPs in matrix multiplication
- Backward and Forward passes FLOPs

FLOPS IN MATRIX MULTIPLICATION

- FLOPS are the number of multiplication and additions

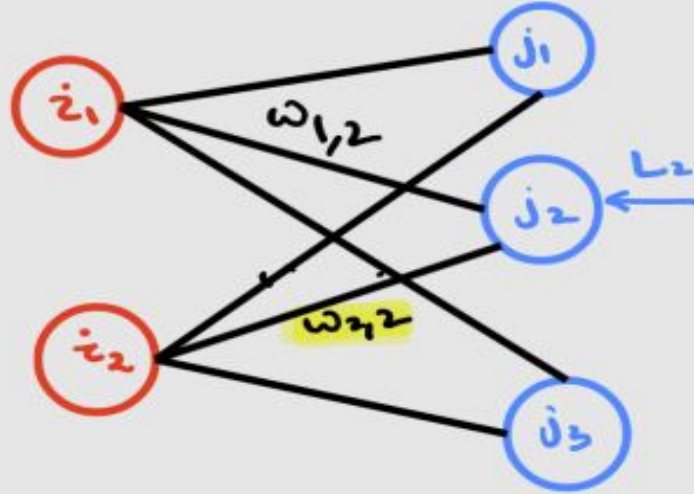


$$N \text{ (multiplications)} + N-1 \text{ (additions)} = 2N-1$$

Repeat this for M times along the rows and K times along the column

$$\text{Total} = MK(2N-1)$$

Backward is twice as expensive as forward



⑥ Add/accumulate.

$$\sum_{i=1}^M \frac{L_{2,i}}{\partial w_{2,2}}$$

$M = \text{mini-batch size}$

$$j_2 = w_{2,2}^{(1)} x_2^{(2)} + w_{1,2}^{(2)} x_1^{(1)}$$

$$\frac{\partial L_2}{\partial i_2} = \frac{\partial L_2}{\partial j_2} \times \frac{\partial j_2}{\partial i_2}$$

$$= \frac{\partial L_2}{\partial j_2} \times w_{2,2} - (2)$$

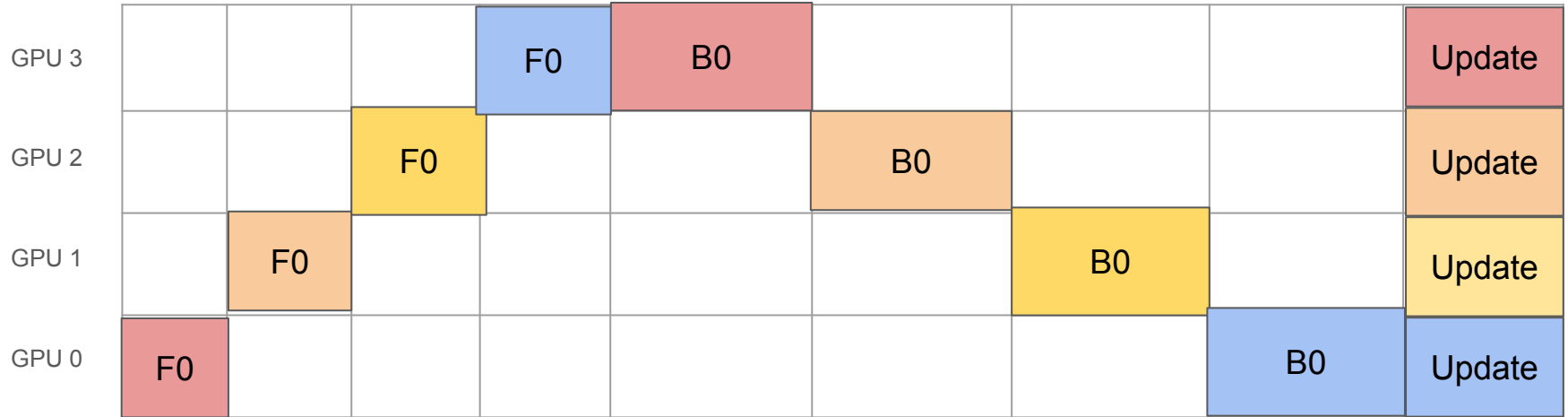
$$\frac{\partial L_{i_3}}{\partial i_2} = \frac{\partial L_1}{\partial i_2} + \frac{\partial L_2}{\partial i_2} + \frac{\partial L_3}{\partial i_2}$$

$$\frac{\partial L_2}{\partial w_{2,2}} = \frac{\partial L_2}{\partial j_2} \frac{\partial j_2}{\partial w_{2,2}} = \frac{\partial L_2}{\partial j_2} i_2$$

⑤ $\hookrightarrow$ From eqn ②
multiply

NAIVE PIPELINE PARALLELISM

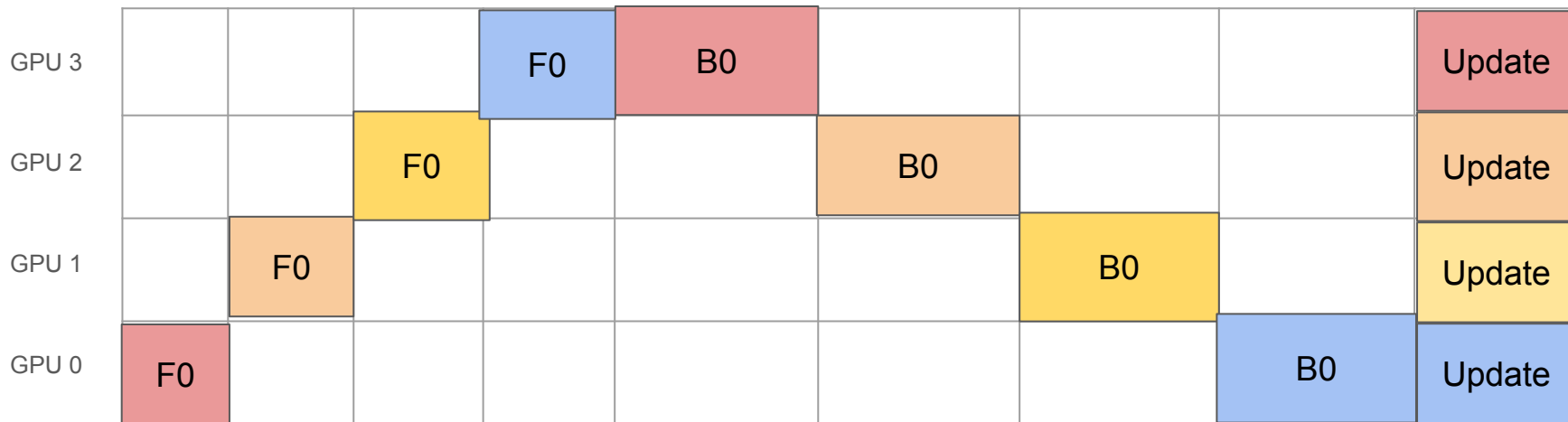
Bubble ratio = $\text{bubble_time} / \text{useful compute time}$



NAIVE PIPELINE PARALLELISM

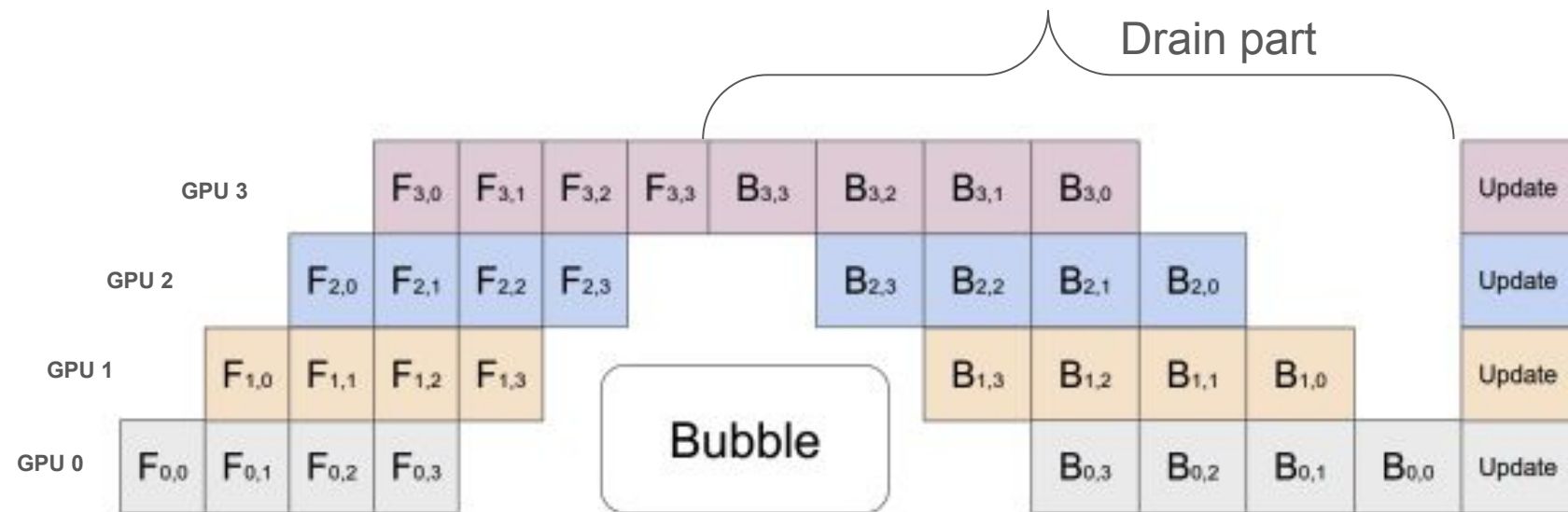
PP: Pipeline parallel size or 4 GPUS

Total forward and backward time = $t_f + t_b$



Bubble: $(PP-1) \cdot (t_f + t_b) / (t_f + t_b) = (PP-1)$

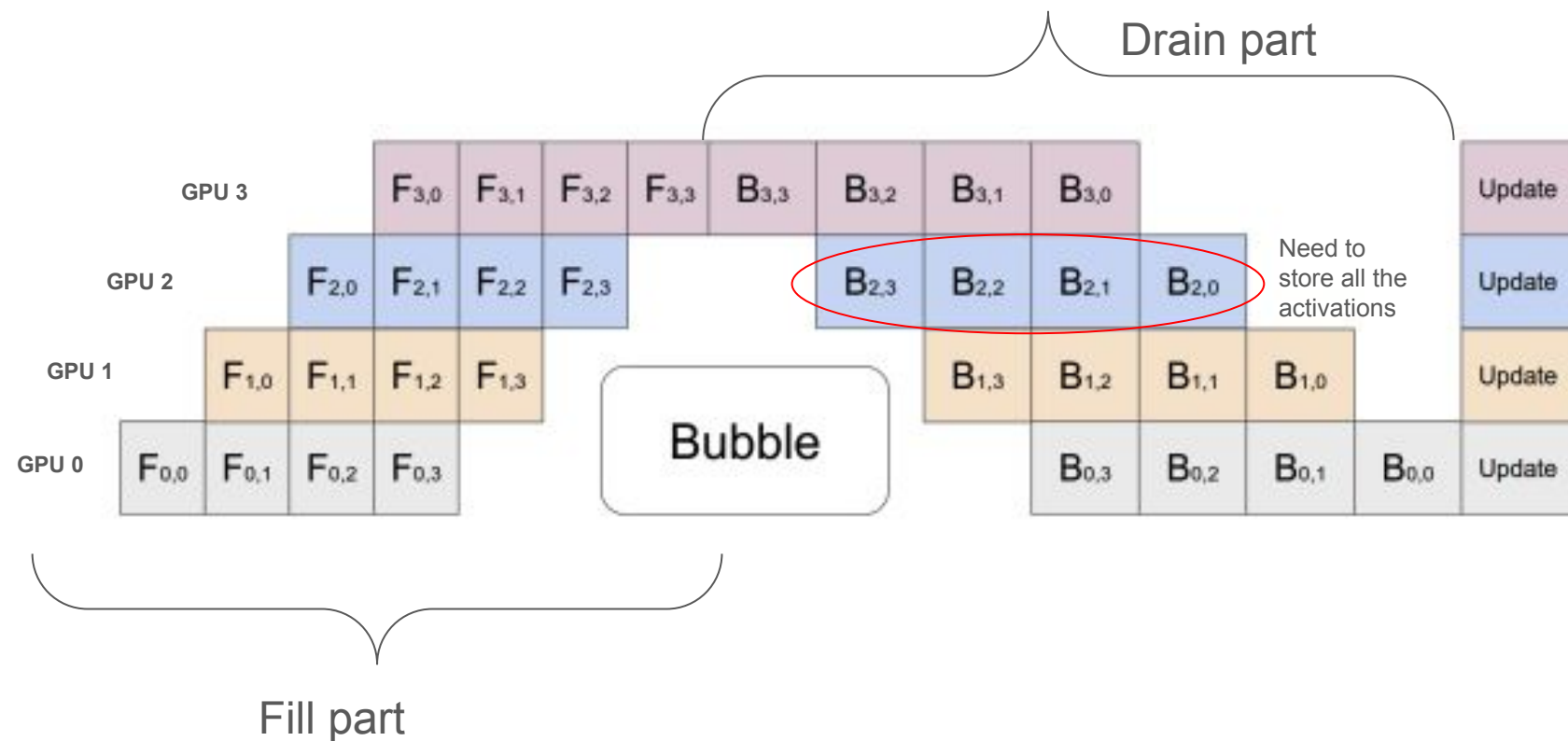
GPiPE (all forward all backward) with mini batches (micro > mini)



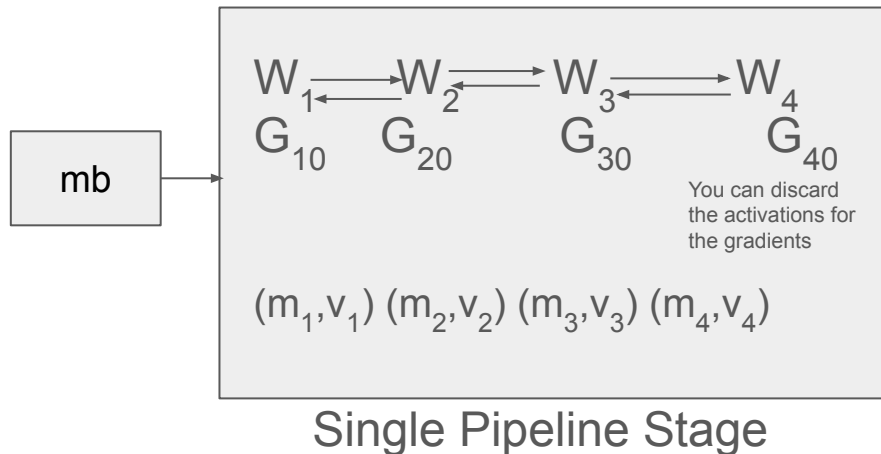
Fill part

Bubble: $(PP-1)(t_f + t_b) / m*(t_f + t_b)$
m is mini batch size

Problems with afab (activation explosion)



Optimizers synchronization



W_i Parameter

G_{ij} Gradient for i and device j

m_i first moment for parameter i

v_i second moment for parameter i

Memory (N parameters)

Weight = $2 * N$ (bf16)

Gradients = $4 * N$ (fp32)

**Optimizers = $4 * N + 4 * N$
(fp32)**

Master weights = $4 * N$ (fp32)

Update parameters (W_1)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

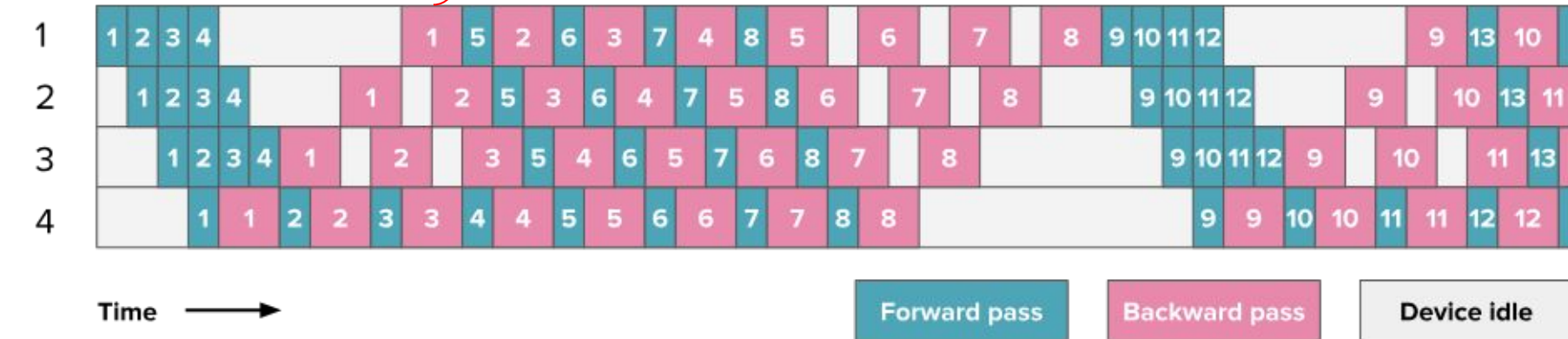
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta'_t = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

1f1b or 1 forward and 1 backward

GPU  Activations for **mini batch 1** can be discarded



Zero Bubble Pipeline Parallelism

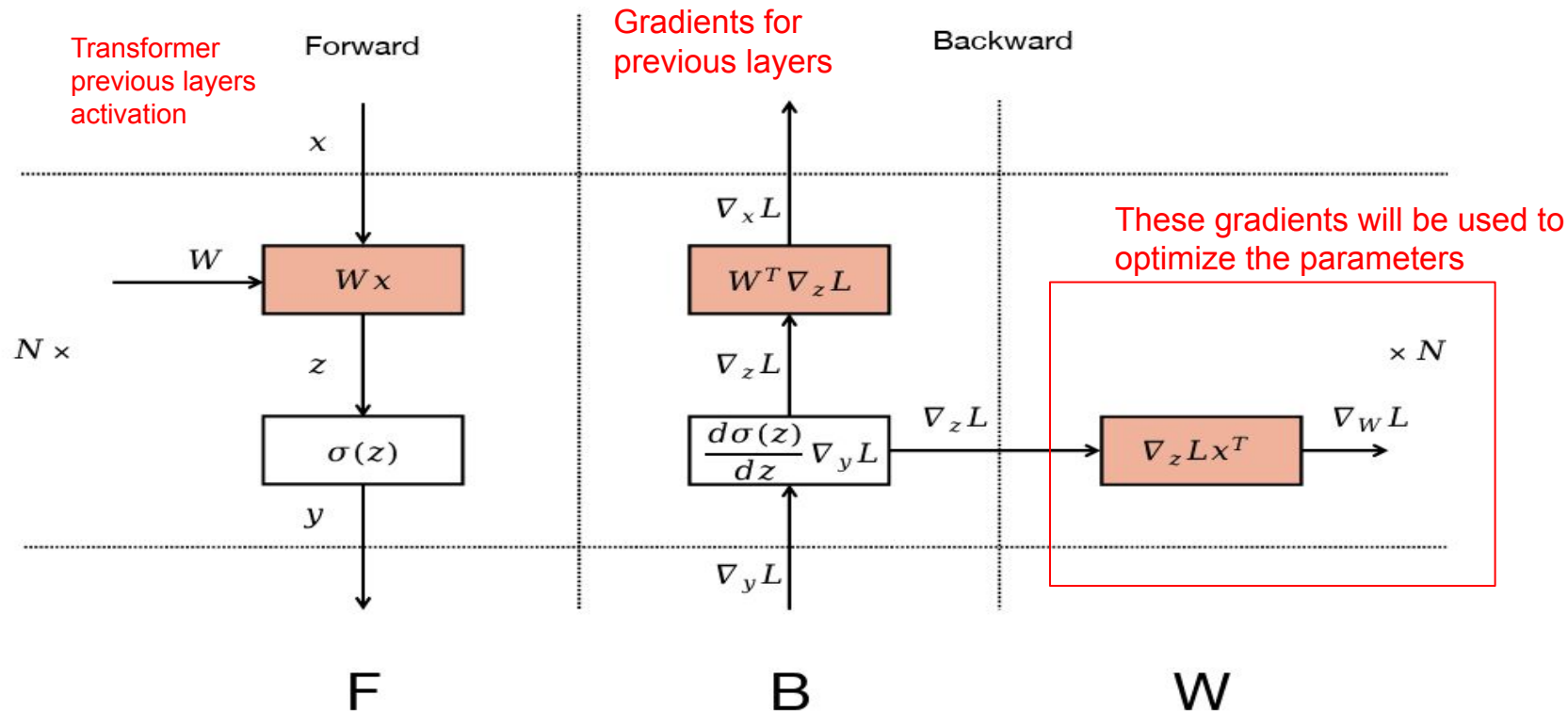
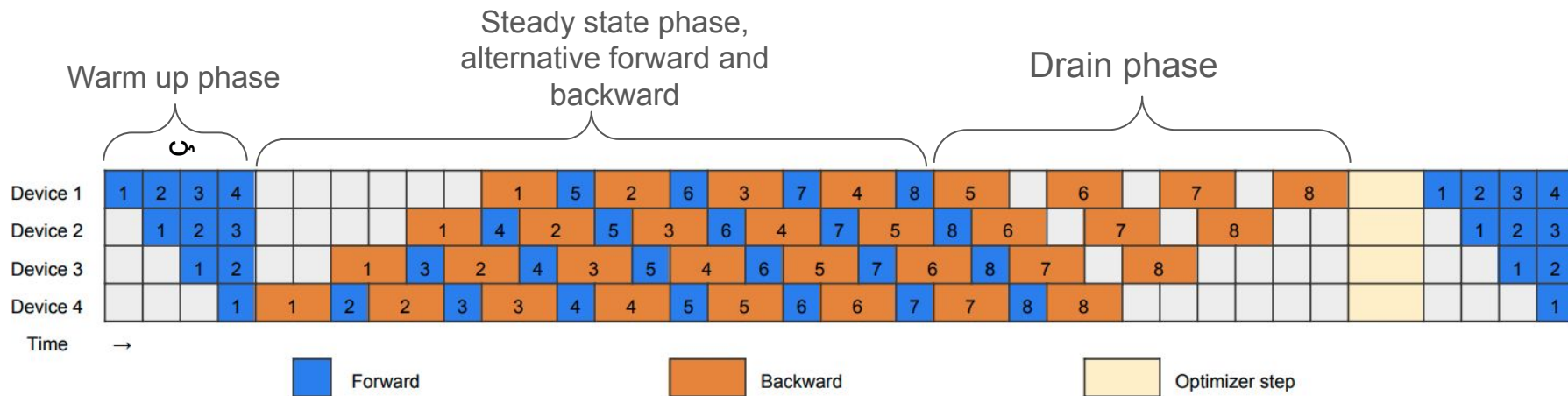


Figure 1: Computation Graph for MLP.

PRIORITY

- Gradients with respect to the **input (B)** goes to previous layers
- Gradients with respect to the **weights (W)** will wait till the optimizer kicks in
- **PRIORITY(grad_input) > PRIORITY(grad_weights)**
 - This will unblock the earlier layers
- **Also, memory of grad_input(scales with batch size, sequence length and hidden dimension) is higher than memory of grad_weights(scales with model size)**

1f1b stages



Decision for scheduling W (grad_weights), B (grad_inputs), F(forward pass)

1. Schedule as many forward passes in the warm up phase (keeping memory in mind)
 - a. This will reduce the bubble size before backward pass starts
 - b. But prioritize going to backward pass for B over forward passes (this is controlled by a binary hyperparameter)

Decision for scheduling W (grad_weights), B (grad_inputs), F(forward pass)

W is used for
optimizer
B goes to previous
layers

1. Schedule as many forward passes in the warm up phase (keeping memory in mind)
 - a. This will reduce the bubble size before backward pass starts
 - b. But prioritize going to backward pass for B over forward passes (this is controlled by a binary hyperparameter)
2. Do one 1F and 1B (optionally do 1W)
 - a. If bubble gap > threshold T_w (time for forward pass)
 - i. Then do 1W
 - ii. If activation memory increases, then do a W to free up that memory

Decision for scheduling W (grad_weights), B (grad_inputs), F(forward pass)

F will reduce bubble but increase activations
B will reduce activations
W will also reduce activations

1. Schedule as many forward passes in the warm up phase (keeping memory in mind)
 - a. This will reduce the bubble size before backward pass starts
 - b. But prioritize going to backward pass for B over forward passes (this is controlled by a **binary hyperparameter**)
2. Do one 1F and 1B (optionally do 1W)
 - a. If bubble gap > threshold T_w
 - i. Then do 1W
 - ii. If activation memory increases, then do a W to free up that memory
3. We use another **binary hyperparameter** to decide if we want to schedule a forward pass and use up all our mini-batches, or free up activations with backward passes 1B or 1W

Trade-off for the binary hyperparameter

- There are two binary hyperparameters
 - One occurs at the start of the pipeline (warm up phase)
 - Here the trade-off starting backward pass B (grad_inputs) or we keep on doing many forward passes (F)
 - If we do forward passes, then we will fill up the bubble (which is good)
 - But it will also increase the activations memory, and we need 1B to free up

Trade-off for the binary hyperparameter

- Another occurs during the pipeline (steady phase interleaving backward and forward pass)
 - Similar trade-offs, whether to fill the pipeline with forward passes or do backward pass and free up activations memory

Schedulers

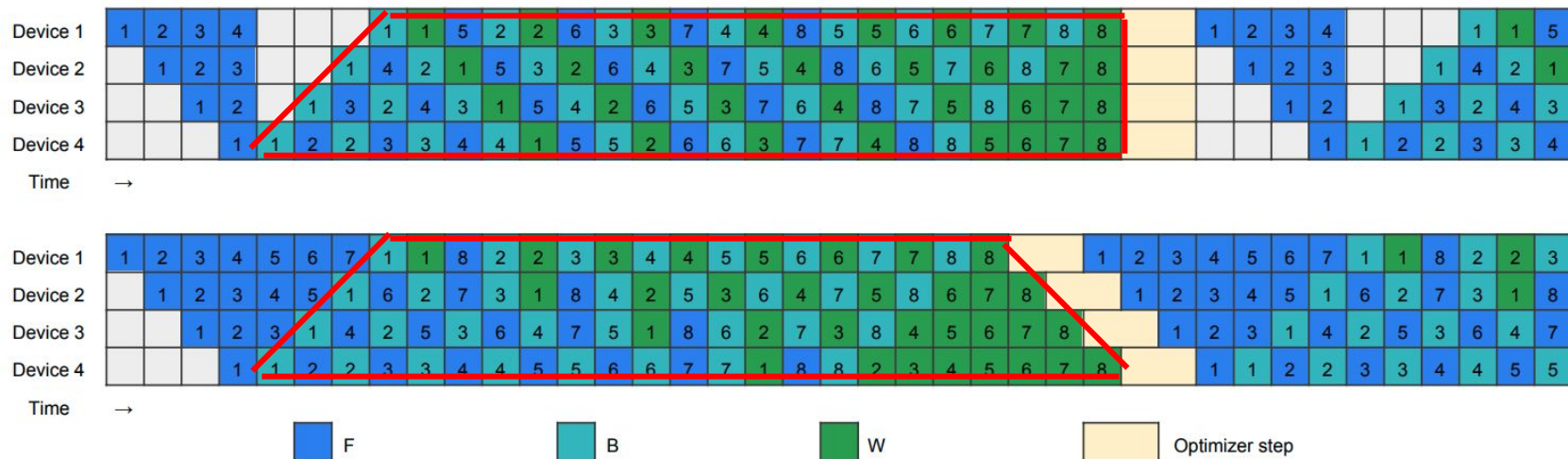
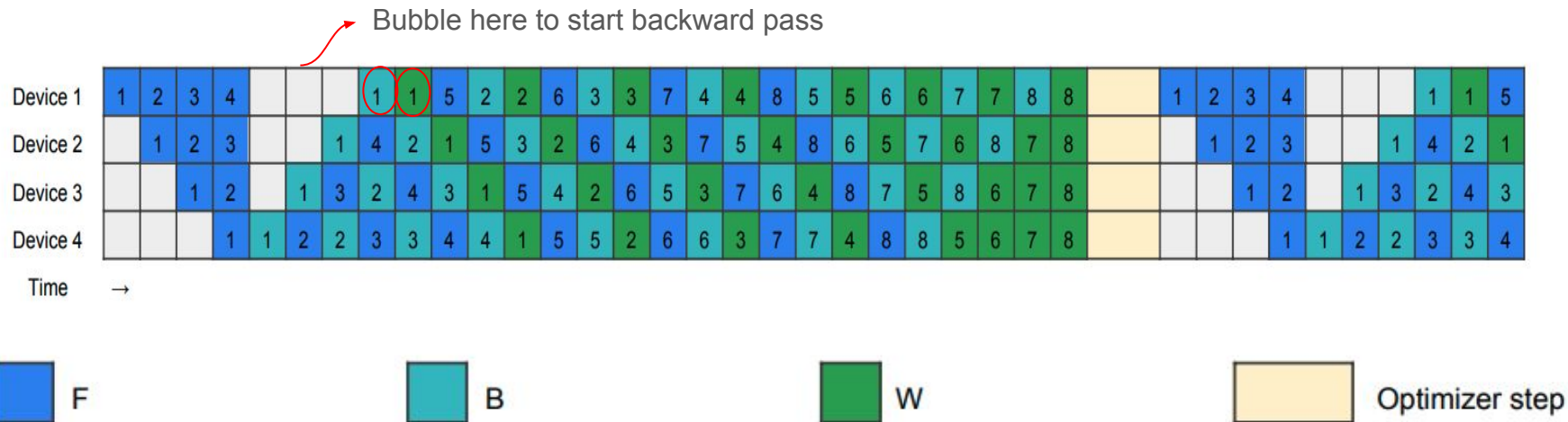


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

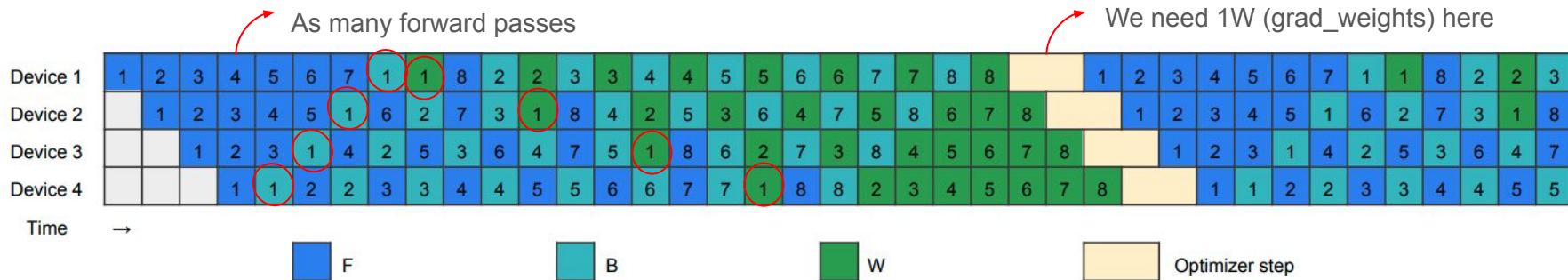
ZB-H1

- Prioritize memory for starting the backward pass B (grad_inputs) early and also backward pass (W or grad_weights) early too so that we can free up activations memory



ZB-H2

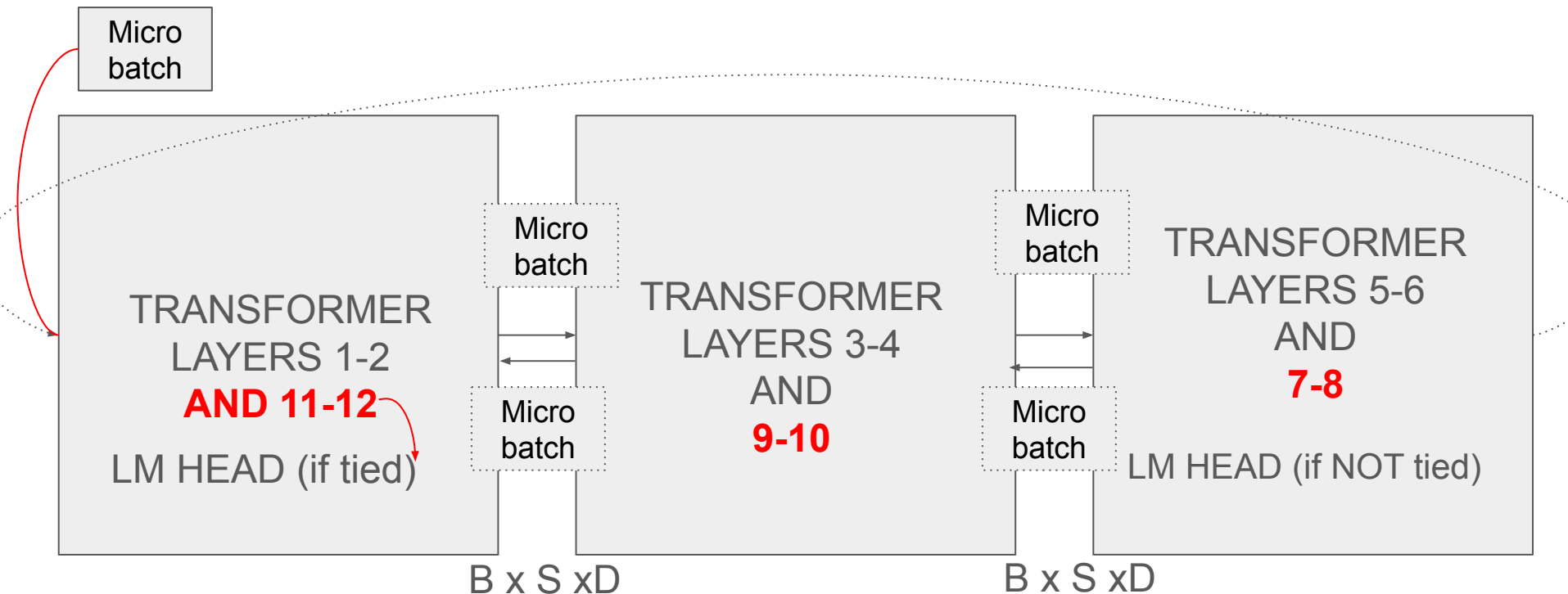
- Prioritize reducing the bubble size
 - As many forward passes as possible before starting backward
 - 1W (grad_weights) are scheduled if the activations memory is high later on the pipeline to accommodate more forward passes



ZB-V Schedule

- The ZB-H2 reduced the bubble size, but it has high activations memory requirement
- Some intuitions
 - More consecutive layers in a GPU leads to more activations memory storage
 - Can we decouple the layers?

PIPELINE PARALLELISM MODEL SPLIT (FORWARD)

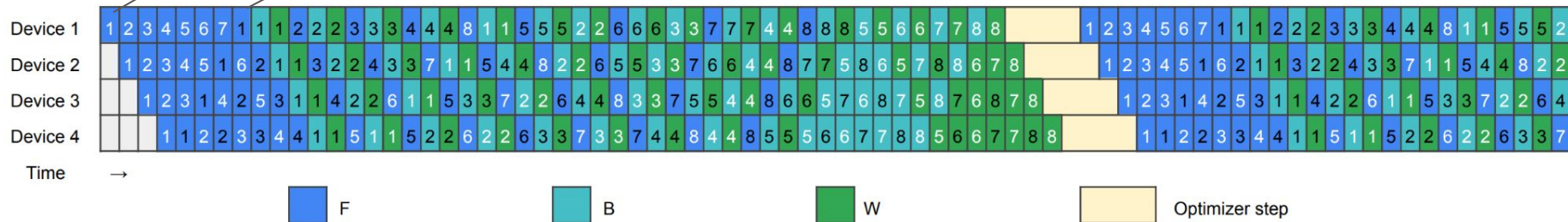


ZB-V SCHEDULE

Notice how the size of the backward has decreased

FIRST LAYERS

LATER LAYERS



PIPELINE PARALLELISM MODEL SPLIT (BACKWARD)

